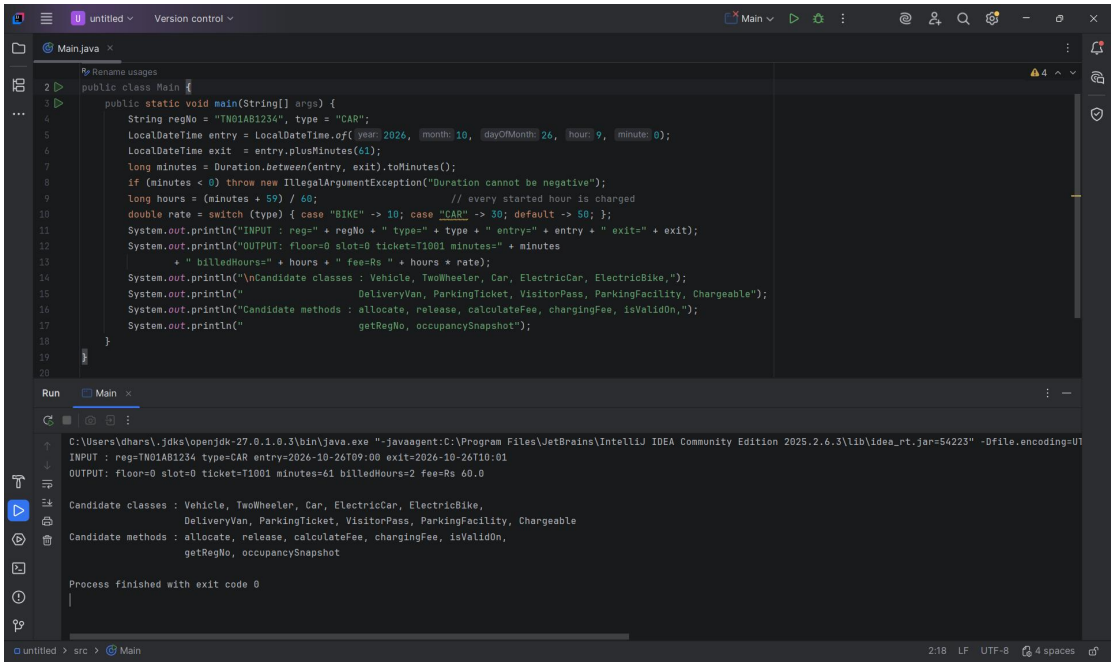
Java Programming Assignment 1

Tasks 1–12

192324059 Dharshini J

Task 1 — Understand the problem and identify objects



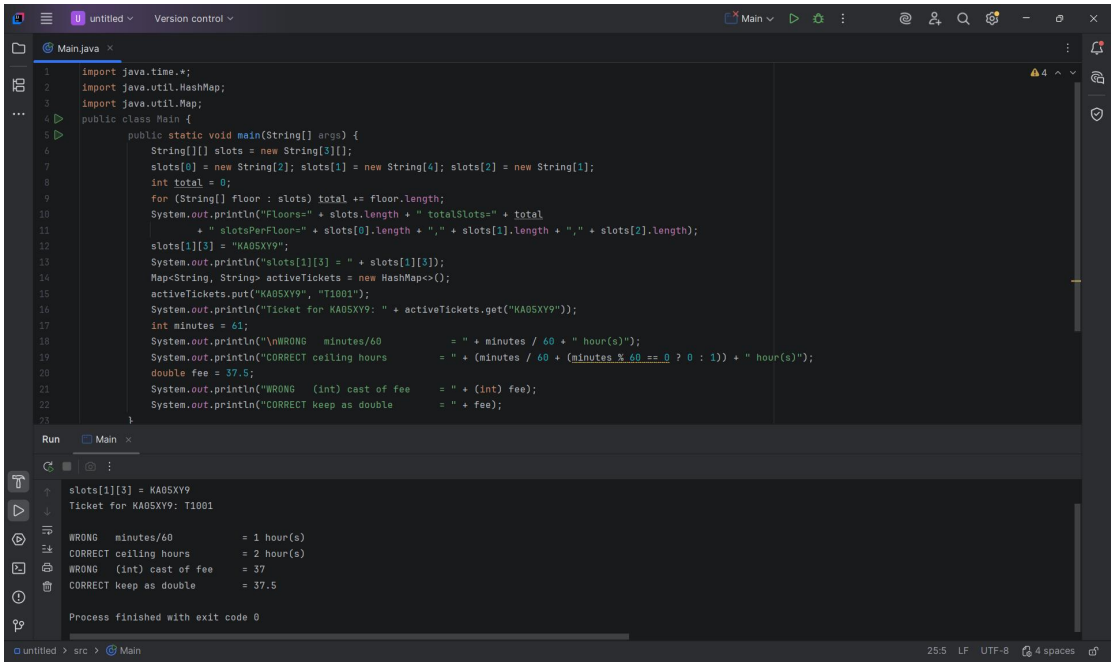
```
public class Main {
    public static void main(String[] args) {
        String regNo = "TN01AB1234", type = "CAR";
        LocalDateTime entry = LocalDateTime.of( year: 2026, month: 10, dayOfMonth: 26, hour: 9, minute: 0);
        LocalDateTime exit  = entry.plusMinutes(61);
        long minutes = Duration.between(entry, exit).toMinutes();
        if (minutes < 0) throw new IllegalArgumentException("Duration cannot be negative");
        long hours = (minutes + 59) / 60;                    // every started hour is charged
        double rate = switch (type) { case "BIKE" -> 10; case "CAR" -> 30; default -> 50; };
        System.out.println("INPUT : reg=" + regNo + " type=" + type + " entry=" + entry + " exit=" + exit);
        System.out.println("OUTPUT: floor=0 slot=0 ticket=T1001 minutes=" + minutes
                + " billedHours=" + hours + " fee=Rs " + hours * rate);
        System.out.println("\nCandidate classes : Vehicle, TwoWheeler, Car, ElectricCar, ElectricBike,");
        System.out.println("                    DeliveryVan, ParkingTicket, VisitorPass, ParkingFacility, Chargeable");
        System.out.println("Candidate methods : allocate, release, calculateFee, chargingFee, isValidOn,");
        System.out.println("                    getRegNo, occupancySnapshot");
    }
}
```

```
C:\Users\dhars\.jdks\openjdk-27.0.1.0.3\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.2.6.3\lib\idea_rt.jar=54223" -Dfile.encoding=UT
INPUT : reg=TN01AB1234 type=CAR entry=2026-10-26T09:00 exit=2026-10-26T10:01
OUTPUT: floor=0 slot=0 ticket=T1001 minutes=61 billedHours=2 fee=Rs 60.0

Candidate classes : Vehicle, TwoWheeler, Car, ElectricCar, ElectricBike,
                    DeliveryVan, ParkingTicket, VisitorPass, ParkingFacility, Chargeable
Candidate methods : allocate, release, calculateFee, chargingFee, isValidOn,
                    getRegNo, occupancySnapshot

Process finished with exit code 0
```

Task 2 — Choose data representation



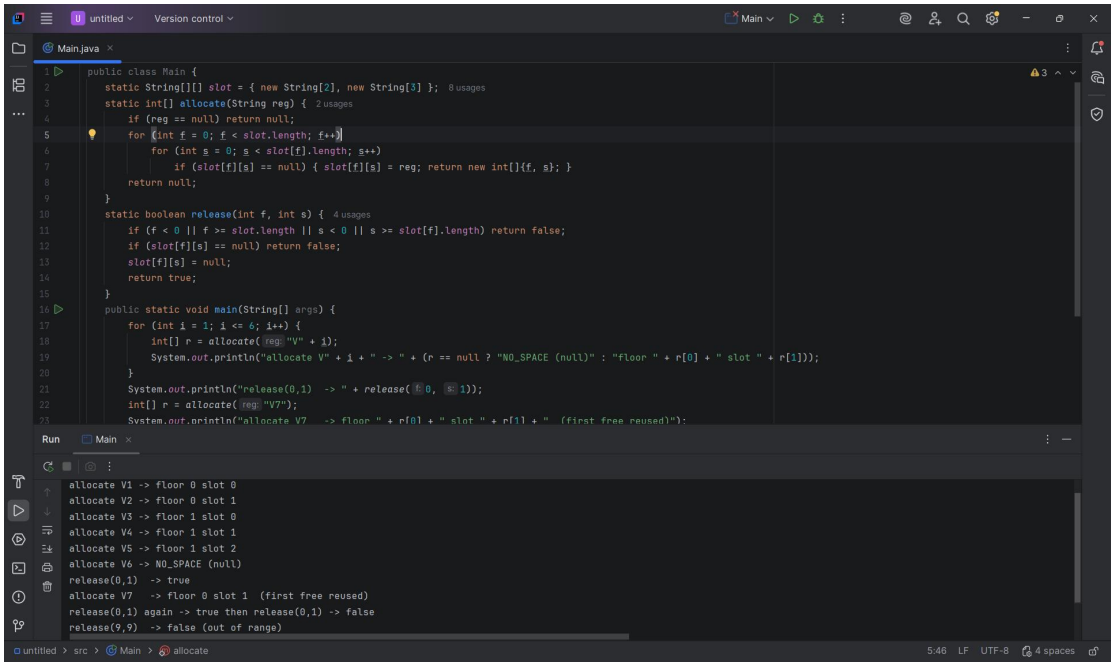
```
import java.time.*;
import java.util.HashMap;
import java.util.Map;
public class Main {
        public static void main(String[] args) {
            String[][] slots = new String[3][];
            slots[0] = new String[2]; slots[1] = new String[4]; slots[2] = new String[1];
            int total = 0;
            for (String[] floor : slots) total += floor.length;
            System.out.println("Floors=" + slots.length + " totalSlots=" + total
                    + " slotsPerFloor=" + slots[0].length + "," + slots[1].length + "," + slots[2].length);
            slots[1][3] = "KA05XY9";
            System.out.println("slots[1][3] = " + slots[1][3]);
            Map<String, String> activeTickets = new HashMap<>();
            activeTickets.put("KA05XY9", "T1001");
            System.out.println("Ticket for KA05XY9: " + activeTickets.get("KA05XY9"));
            int minutes = 61;
            System.out.println("\nWRONG   minutes/60           = " + minutes / 60 + " hour(s)");
            System.out.println("CORRECT ceiling hours        = " + (minutes / 60 + (minutes % 60 == 0 ? 0 : 1)) + " hour(s)");
            double fee = 37.5;
            System.out.println("WRONG   (int) cast of fee    = " + (int) fee);
            System.out.println("CORRECT keep as double       = " + fee);
        }
```

```
slots[1][3] = KA05XY9
Ticket for KA05XY9: T1001

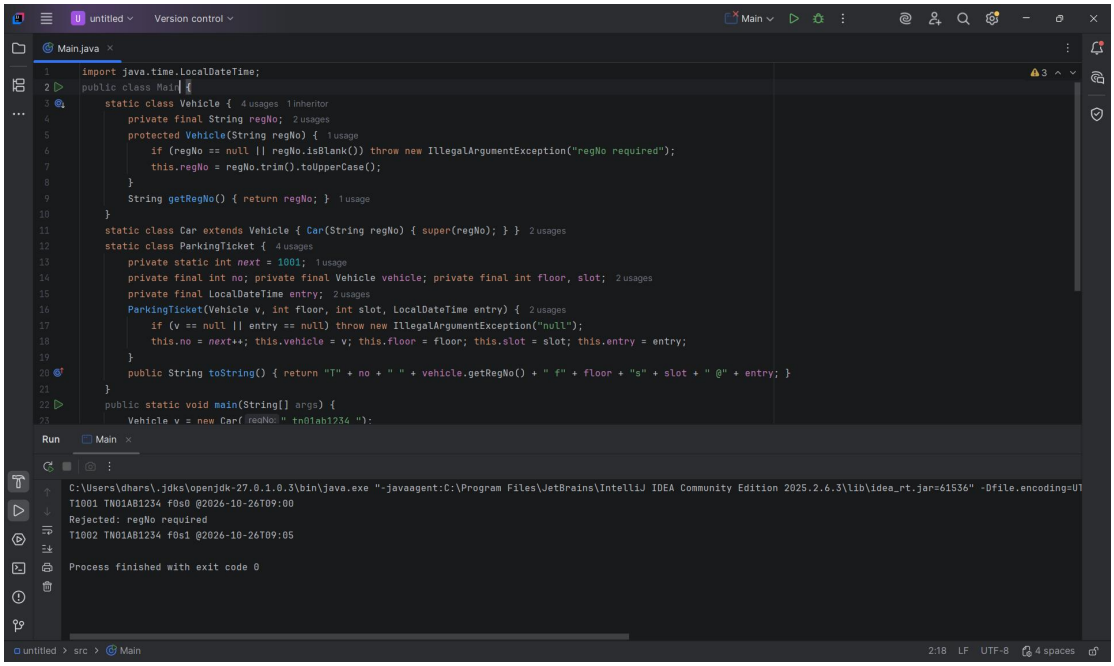
WRONG   minutes/60           = 1 hour(s)
CORRECT ceiling hours        = 2 hour(s)
WRONG   (int) cast of fee    = 37
CORRECT keep as double       = 37.5

Process finished with exit code 0
```

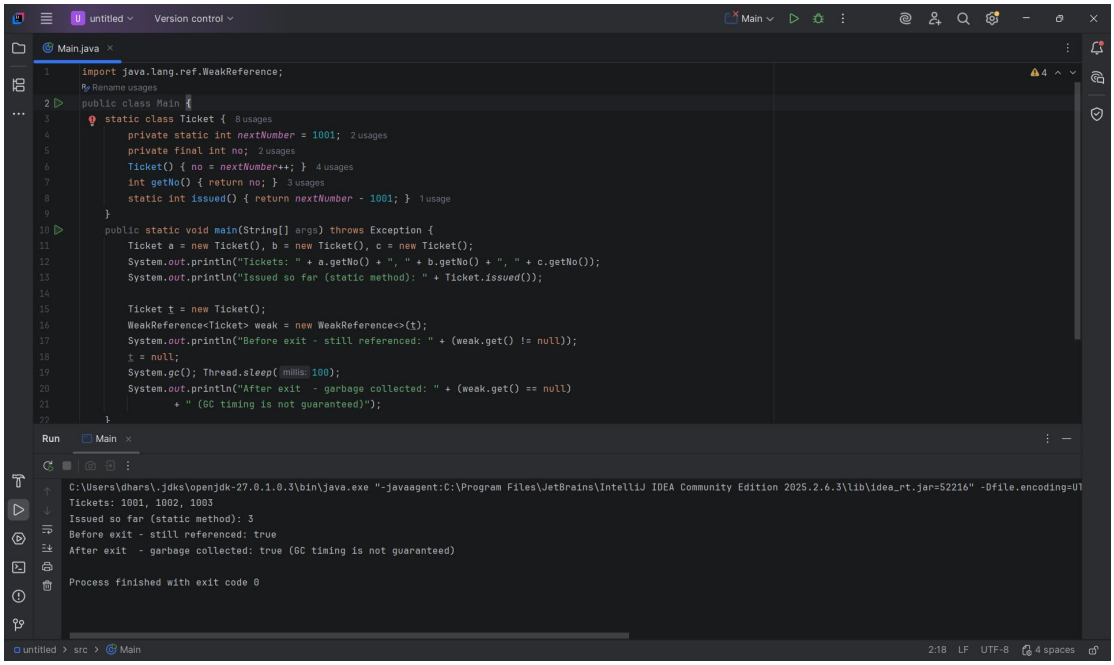
Task 3 — Control logic and pseudocode



Task 4 — Classes, objects and constructors



Task 5 — Static members and object life cycle



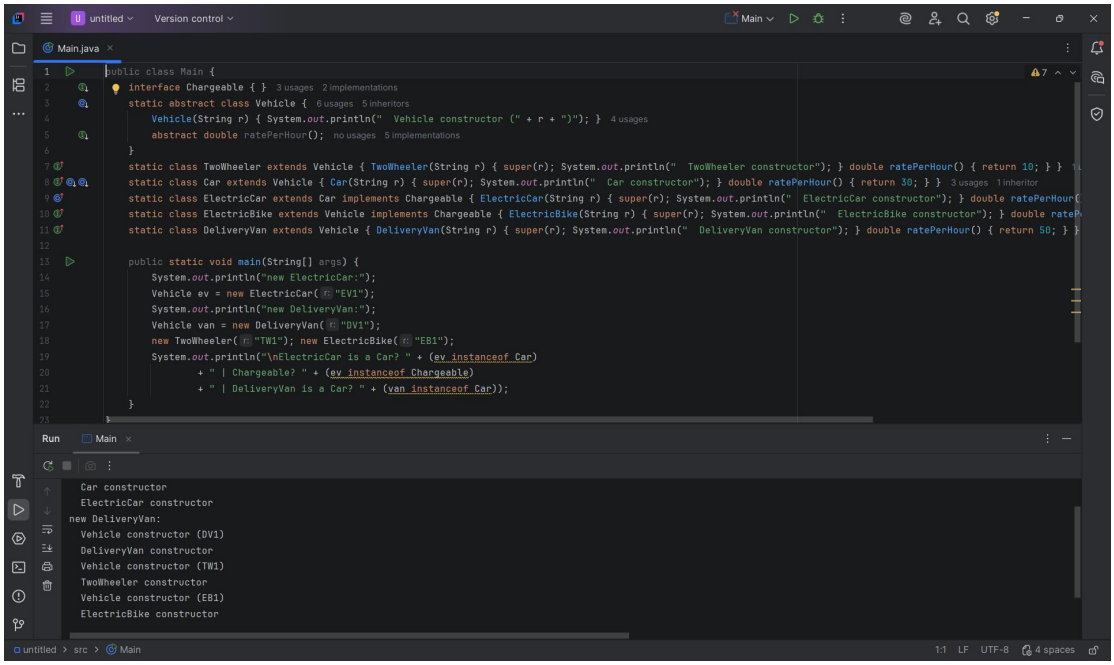
```
import java.lang.ref.WeakReference;
public class Main {
    static class Ticket {
        private static int nextNumber = 1001;
        private final int no;
        Ticket() { no = nextNumber++; }
        int getNo() { return no; }
        static int issued() { return nextNumber - 1001; }
    }
    public static void main(String[] args) throws Exception {
        Ticket a = new Ticket(), b = new Ticket(), c = new Ticket();
        System.out.println("Tickets: " + a.getNo() + ", " + b.getNo() + ", " + c.getNo());
        System.out.println("Issued so far (static method): " + Ticket.issued());

        Ticket t = new Ticket();
        WeakReference<Ticket> weak = new WeakReference<>(t);
        System.out.println("Before exit - still referenced: " + (weak.get() != null));
        t = null;
        System.gc(); Thread.sleep(millis: 100);
        System.out.println("After exit  - garbage collected: " + (weak.get() == null)
                + " (GC timing is not guaranteed)");
    }
}
```

```
C:\Users\dhars\.jdks\openjdk-27.0.1.0.3\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.2.6.3\lib\idea_rt.jar=52216" -Dfile.encoding=UT
Tickets: 1001, 1002, 1003
Issued so far (static method): 3
Before exit - still referenced: true
After exit  - garbage collected: true (GC timing is not guaranteed)

Process finished with exit code 0
```

Task 6 — Inheritance hierarchy

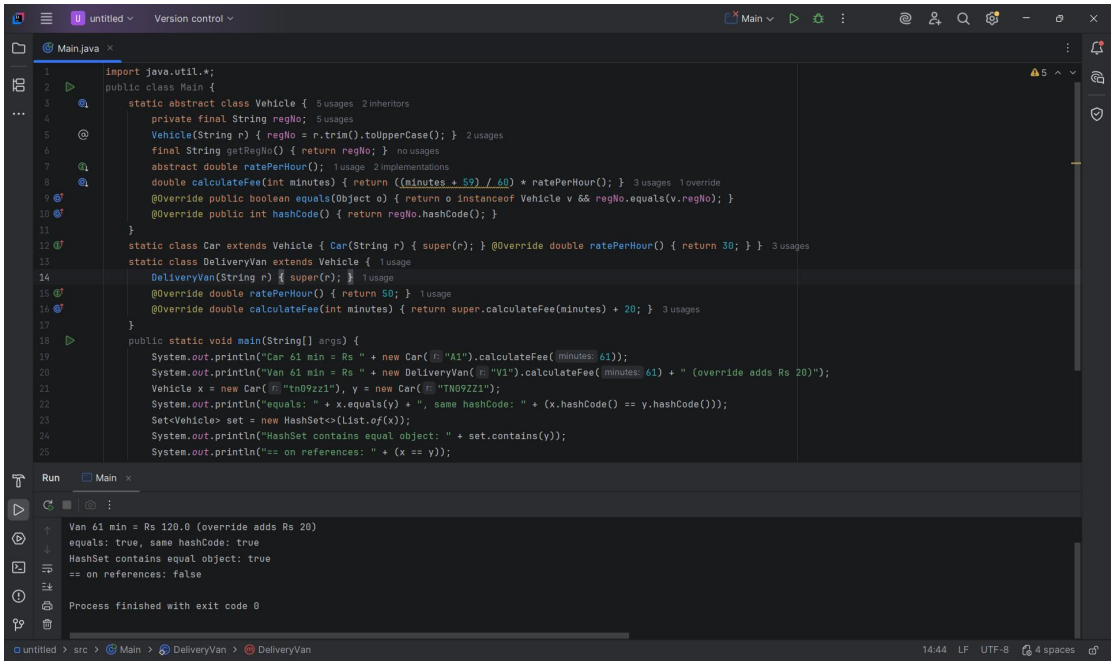


```
public class Main {
    interface Chargeable { }
    static abstract class Vehicle {
        Vehicle(String r) { System.out.println("  Vehicle constructor (" + r + ")"); }
        abstract double ratePerHour();
    }
    static class TwoWheeler extends Vehicle { TwoWheeler(String r) { super(r); System.out.println("  TwoWheeler constructor"); } double ratePerHour() { return 10; } }
    static class Car extends Vehicle { Car(String r) { super(r); System.out.println("  Car constructor"); } double ratePerHour() { return 30; } }
    static class ElectricCar extends Car implements Chargeable { ElectricCar(String r) { super(r); System.out.println("  ElectricCar constructor"); } double ratePerHour(
    static class ElectricBike extends Vehicle implements Chargeable { ElectricBike(String r) { super(r); System.out.println("  ElectricBike constructor"); } double rateP
    static class DeliveryVan extends Vehicle { DeliveryVan(String r) { super(r); System.out.println("  DeliveryVan constructor"); } double ratePerHour() { return 50; } }

    public static void main(String[] args) {
        System.out.println("new ElectricCar:");
        Vehicle ev = new ElectricCar( r: "EV1");
        System.out.println("new DeliveryVan:");
        Vehicle van = new DeliveryVan( r: "DV1");
        new TwoWheeler( r: "TW1"); new ElectricBike( r: "EB1");
        System.out.println("\nElectricCar is a Car? " + (ev instanceof Car)
                + " | Chargeable? " + (ev instanceof Chargeable)
                + " | DeliveryVan is a Car? " + (van instanceof Car));
    }
}
```

```
  Car constructor
  ElectricCar constructor
new DeliveryVan:
  Vehicle constructor (DV1)
  DeliveryVan constructor
  Vehicle constructor (TW1)
  TwoWheeler constructor
  Vehicle constructor (EB1)
  ElectricBike constructor
```

Task 7 — Override and specialise behaviour

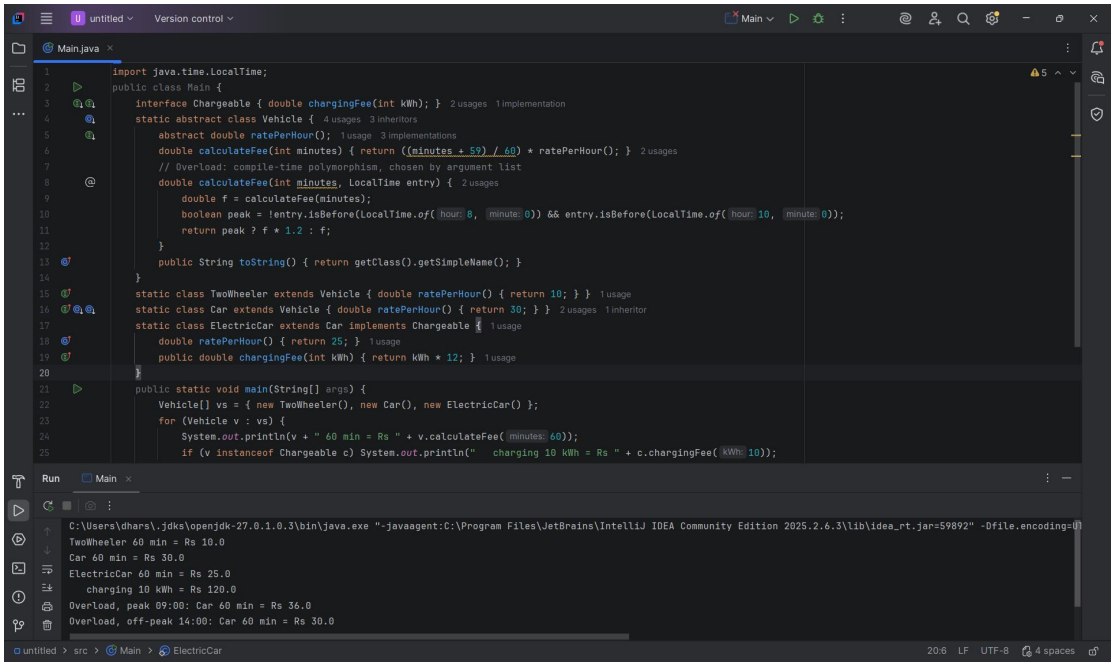


```
import java.util.*;
public class Main {
    static abstract class Vehicle {  5 usages  2 inheritors
        private final String regNo;  5 usages
        Vehicle(String r) { regNo = r.trim().toUpperCase(); }  2 usages
        final String getRegNo() { return regNo; }  no usages
        abstract double ratePerHour();  1 usage  2 implementations
        double calculateFee(int minutes) { return ((minutes + 59) / 60) * ratePerHour(); }  3 usages  1 override
        @Override public boolean equals(Object o) { return o instanceof Vehicle v && regNo.equals(v.regNo); }
        @Override public int hashCode() { return regNo.hashCode(); }
    }
    static class Car extends Vehicle { Car(String r) { super(r); } @Override double ratePerHour() { return 30; } }  3 usages
    static class DeliveryVan extends Vehicle {  1 usage
        DeliveryVan(String r) { super(r); }  1 usage
        @Override double ratePerHour() { return 50; }  1 usage
        @Override double calculateFee(int minutes) { return super.calculateFee(minutes) + 20; }  3 usages
    }
    public static void main(String[] args) {
        System.out.println("Car 61 min = Rs " + new Car( r: "A1").calculateFee( minutes: 61));
        System.out.println("Van 61 min = Rs " + new DeliveryVan( r: "V1").calculateFee( minutes: 61) + " (override adds Rs 20)");
        Vehicle x = new Car( r: "tn09zz1"), y = new Car( r: "TN09ZZ1");
        System.out.println("equals: " + x.equals(y) + ", same hashCode: " + (x.hashCode() == y.hashCode()));
        Set<Vehicle> set = new HashSet<>(List.of(x));
        System.out.println("HashSet contains equal object: " + set.contains(y));
        System.out.println("== on references: " + (x == y));
```

```
Van 61 min = Rs 120.0 (override adds Rs 20)
equals: true, same hashCode: true
HashSet contains equal object: true
== on references: false

Process finished with exit code 0
```

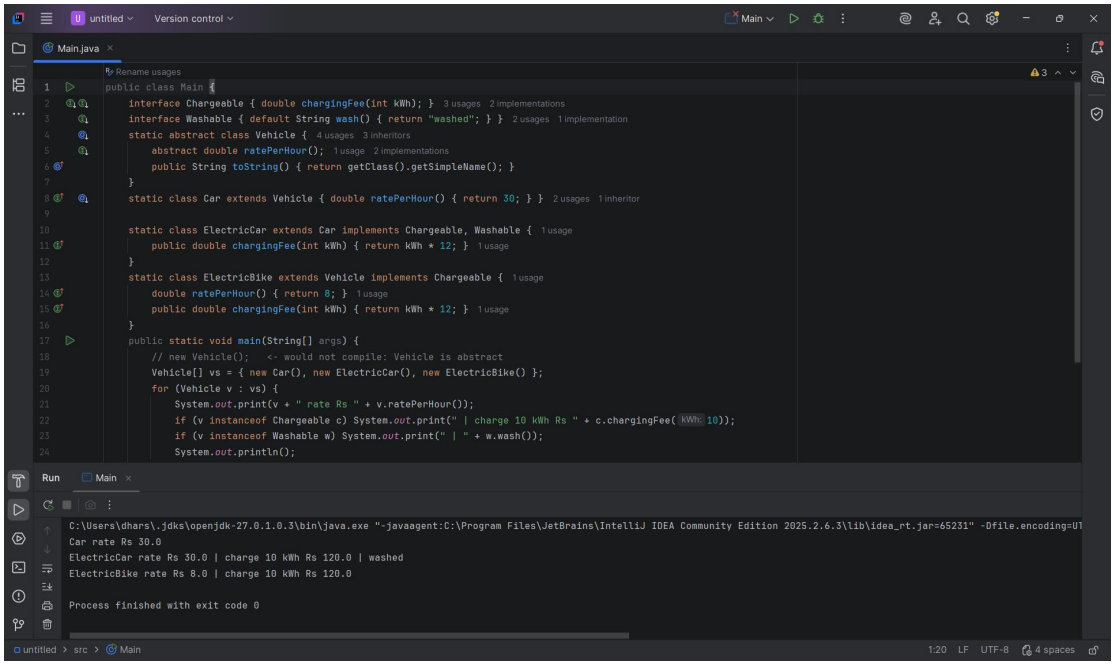
Task 8 — Polymorphism



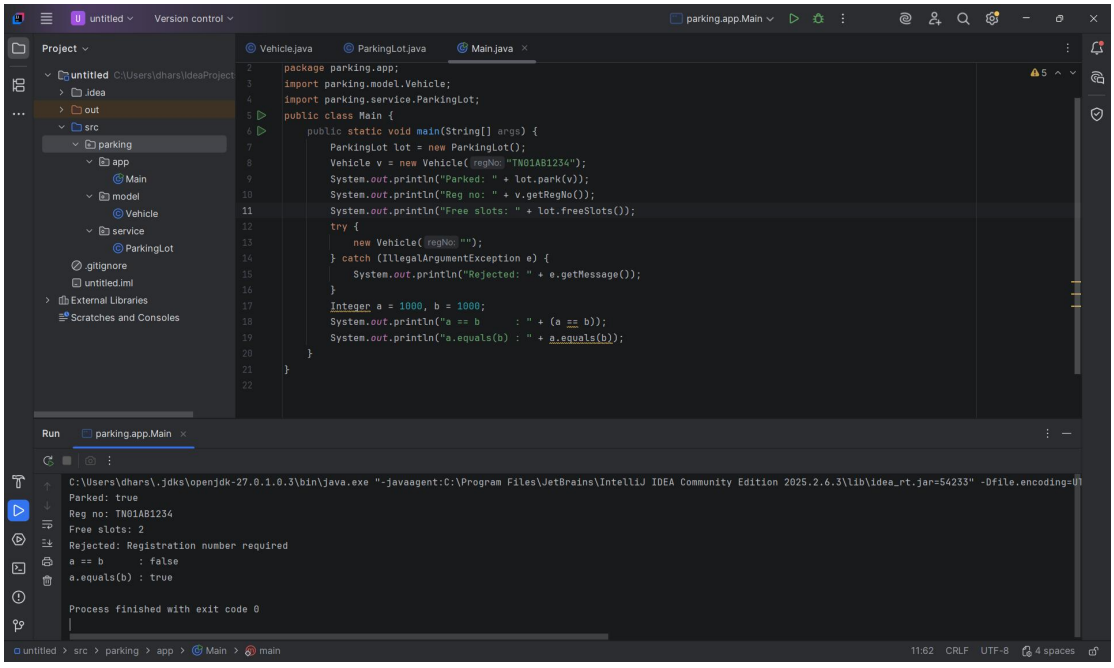
```
import java.time.LocalTime;
public class Main {
    interface Chargeable { double chargingFee(int kWh); }  2 usages  1 implementation
    static abstract class Vehicle {  4 usages  3 inheritors
        abstract double ratePerHour();  1 usage  3 implementations
        double calculateFee(int minutes) { return ((minutes + 59) / 60) * ratePerHour(); }  2 usages
        // Overload: compile-time polymorphism, chosen by argument list
        double calculateFee(int minutes, LocalTime entry) {  2 usages
            double f = calculateFee(minutes);
            boolean peak = !entry.isBefore(LocalTime.of( hour: 8,  minute: 0)) && entry.isBefore(LocalTime.of( hour: 10,  minute: 0));
            return peak ? f * 1.2 : f;
        }
        public String toString() { return getClass().getSimpleName(); }
    }
    static class TwoWheeler extends Vehicle { double ratePerHour() { return 10; } }  1 usage
    static class Car extends Vehicle { double ratePerHour() { return 30; } }  2 usages  1 inheritor
    static class ElectricCar extends Car implements Chargeable {  1 usage
        double ratePerHour() { return 25; }  1 usage
        public double chargingFee(int kWh) { return kWh * 12; }  1 usage
    }
    public static void main(String[] args) {
        Vehicle[] vs = { new TwoWheeler(), new Car(), new ElectricCar() };
        for (Vehicle v : vs) {
            System.out.println(v + " 60 min = Rs " + v.calculateFee( minutes: 60));
            if (v instanceof Chargeable c) System.out.println("   charging 10 kWh = Rs " + c.chargingFee( kWh: 10));
```

```
C:\Users\dhars\.jdks\openjdk-27.0.1.0.3\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.2.6.3\lib\idea_rt.jar=59892" -Dfile.encoding=U
TwoWheeler 60 min = Rs 10.0
Car 60 min = Rs 30.0
ElectricCar 60 min = Rs 25.0
   charging 10 kWh = Rs 120.0
Overload, peak 09:00: Car 60 min = Rs 36.0
Overload, off-peak 14:00: Car 60 min = Rs 30.0
```

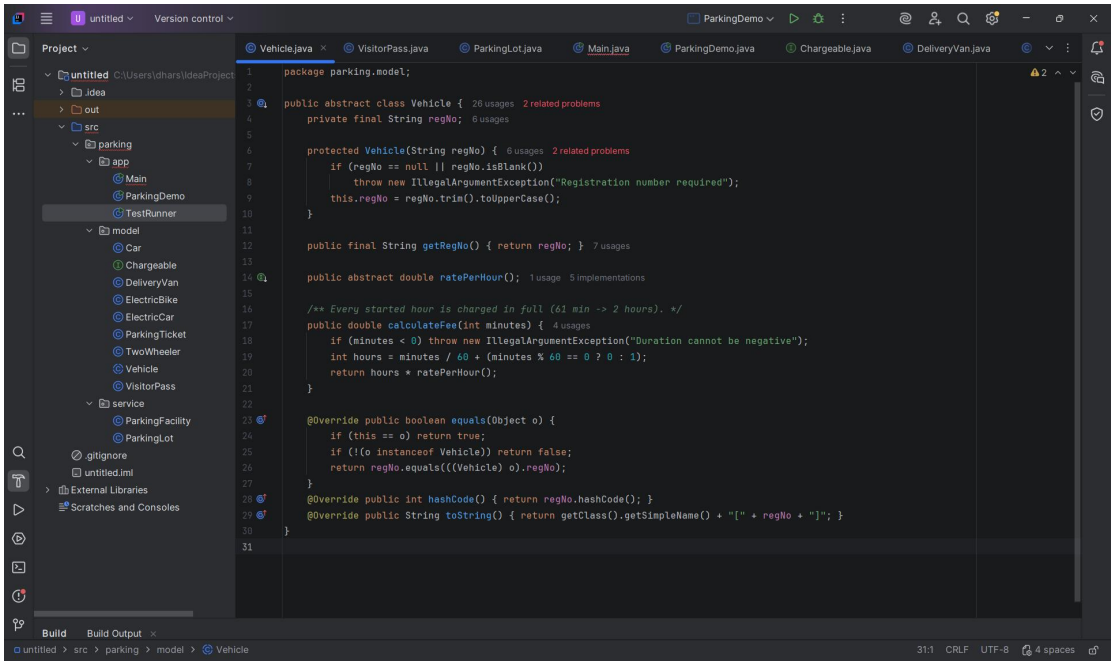
Task 9 — Abstraction and interfaces



Task 10 — Packages and encapsulation



Task 11 — Implement, test and defend



Task 12 — Re-evaluate and adapt

